15

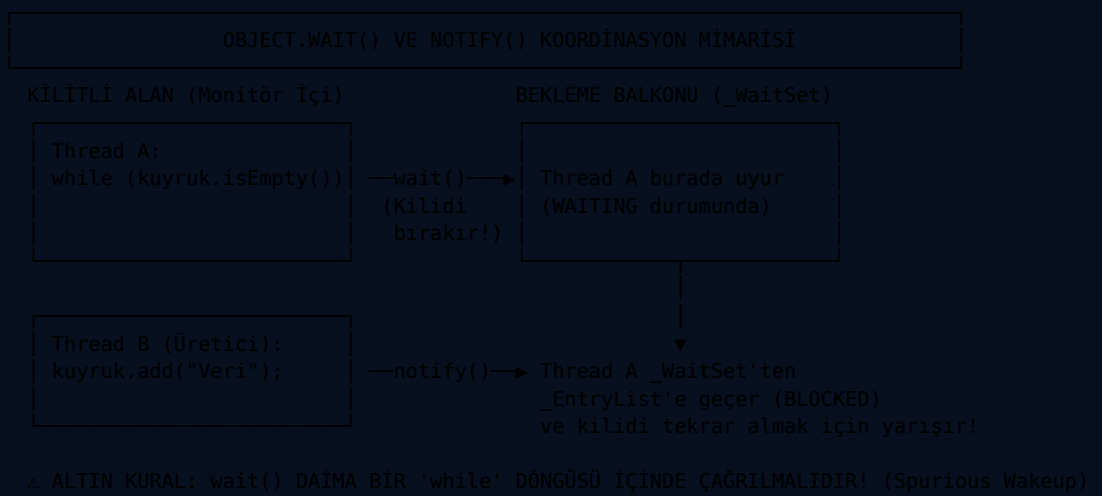
BÖLÜM: KLASIK SENKRONIZASYON, MONITÖRLER & KILITLER | MODULE: CLASSICAL SYNCHRONIZATION, MONITORS & LOCKS

Thread'ler Arası İletişim: wait(), notify(), notifyAll() & Kaçan Sinyal (Missed Signal)

Inter-Thread Signaling: wait(), notify(), notifyAll() & The Missed Signal Hazard

⚡ Bellek & Eşzamanlılık Akış Şeması

⚡ Concurrency & Memory Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir restoranda müşteri (Tüketici) masaya oturuyor. Garsona 'Bana yemek getir' diyor. Mutfakta yemek yoksa müşteri masayı meşgul etmez (wait() çağırıp kilidi bırakır) ve kenardaki bekleme salonuna geçer. Aşçı (Üretici) yemeği pişirdiği anda 'Yemek hazır!' diye zili çalar (notify()). Müşteri zil sesini duyunca hemen gidip masaya oturmaz; önce tabağa bakar: 'Gerçekten yemek geldi mi, yoksa garson yanlışlıkla mı zile bastı?' (while döngüsü). Eğer yemek varsa yer; yoksa tekrar beklemeye geçer!

🇬🇧 ENGLISH (ELI5)

Imagine a patron at a diner. If the soup pot is empty, the customer cannot sit occupying the single service stool. They step aside to the lounge via `wait()`, atomically relinquishing the stool. When the chef brews fresh soup, the chef rings the service bell via `notify()`. The patron awakes, but does NOT blindly eat. They inspect the pot inside a `while` loop: 'Is there soup, or did another patron grab it first, or did lightning ring the bell (Spurious Wakeup)?' If soup exists, they eat; otherwise, they wait again.

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

wait/notify Koordinasyonunda 3 Hayati Kural: 1) Neden synchronized içinde olmalıdır?: `wait()` veya `notify()` kilitsiz çağrılırsa JVM anında `IllegalMonitorStateException` fırlatır; çünkü paylaşılan durumu kontrol etmek ve sinyal göndermek atomik bir bütün olmak zorundadır. 2) Spurious Wakeup (Sahte Uyanma): İşletim sistemi sinyalleri veya kernel kesintileri yüzünden bir thread hiçbir `notify()` çağrılmasa dahi `wait()`ten uyanabilir. Bu yüzden `if (condition) wait();` yazmak ölümcül bir hatadır; DAİMA `while (condition) wait();` yazılmalıdır! 3) notify() vs notifyAll(): `notify()` rastgele tek bir thread uyandırır; uyandırılan thread işi yapamazsa tüm sistem sonsuza dek kilitlenebilir (Deadlock / Missed Signal). Bu yüzden özel bir optimizasyon gerekmiyorsa her zaman `notifyAll()` kullanılmalıdır.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

The Mechanics of Condition Signaling: 1) Prerequisite Monitor Ownership: Invoking `wait()` or `notify()` outside of a synchronized block targeting that exact object immediately throws `IllegalMonitorStateException`. 2) Spurious Wakeups: Under POSIX threading implementations, waiting threads can awaken absent any explicit notification due to OS-level signal handling. Guarding `wait()` with an `if` statement causes race conditions where unverified invariants execute; `while (!condition) { wait(); }` is mathematically required. 3) The `notify()` vs `notifyAll()` Dilemma: `notify()` signals an arbitrary thread from `_WaitSet`. If that thread is unable to satisfy state requirements, the notification is lost, hanging the entire pool. `notifyAll()` broadcasts to all waiting threads, shifting them to `_EntryList`.

⚡ Modern Java 25 LTS Kodu

⚡ Modern Java 25 Implementation

```java
// Java 25: Klasik Producer-Consumer (Üretici-Tüketici) Kuyruğu
import java.util.LinkedList;
import java.util.Queue;

public class BoundedBlockingQueue<T> {
    private final Queue<T> queue = new LinkedList<>();
    private final int capacity;

    public BoundedBlockingQueue(int capacity) {
        this.capacity = capacity;
    }

    public synchronized void put(T item) throws InterruptedException {
        // ⚠ DAİMA WHILE DÖNGÜSÜ! Spurious Wakeup koruması
        while (queue.size() == capacity) {
            wait(); // Kuyruk dolu, yer açılana kadar kilidi bırakıp bekle
        }
        queue.add(item);
        notifyAll(); // Tüketicilere haber ver: Veri geldi!
    }

    public synchronized T take() throws InterruptedException {
        while (queue.isEmpty()) {
            wait(); // Kuyruk boş, veri gelene kadar bekle
        }
        T item = queue.remove();
        notifyAll(); // Üreticilere haber ver: Yer açıldı!
        return item;
    }
}
```

⚠️ Deadlock, Race Condition & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE TEHLİKE

Mülakat Tuzağı: Mülakatçı 'Neden if yerine while kullanıyoruz?' diye sorarsa: '1) Sahte uyanmalar (Spurious Wakeup) yüzünden thread sinyal olmadan uyanabilir. 2) notifyAll çağrıldığında uyandırılan diğer bir thread araya girip kuyruktaki elemanı kapmış olabilir. Bu yüzden uyanan her thread koşulu tekrar doğrulamak zorundadır!' denmelidir.

⚠️ Concurrency Hazard & Interview Trap

🇬🇧 INTERVIEW TRAP & CONCURRENCY HAZARD

Mandatory Interview Answer: 'Why must wait() always sit in a while loop rather than an if block?' Answer: '1) Spurious wakeups allow threads to awaken without signals. 2) When notifyAll awakens multiple threads, another thread may race ahead and consume the state first. The awakening thread must re-evaluate the invariant before proceeding.'

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- wait(), notify() ve notifyAll() sadece synchronized blok içinde çağrılabilir.
- Sahte uyanmalara (Spurious Wakeup) karşı wait() daima while döngüsü içinde olmalıdır.
- Kaçan sinyal (Missed Signal) hatalarını önlemek için varsayılan olarak notifyAll seçilir.

🎯 Key Takeaways for the Interview

- wait() and notify() require active monitor ownership; otherwise IllegalMonitorStateException.
- Always guard wait() with while(!condition) to defend against spurious wakeups.
- Favor notifyAll() over notify() to prevent lost signal deadlocks.